# Simulazione del Comportamento di Stormi

Autori del progetto:

Milazzo Leandro, Milia Raffaele, Morselli Alessandro, Oliverio Luca

Data ultima modifica:

03/07/2025

## Sommario

Tema del progetto e introduzione ..... **Errore. Il segnalibro non è definito.**

Librerie esterne da installare ..... **Errore. Il segnalibro non è definito.**

Istruzioni per eseguire il programma ..... **Errore. Il segnalibro non è definito.**

Parametri di input e formato output ..... **Errore. Il segnalibro non è definito.**

Interpretazione dei risultati ..... **Errore. Il segnalibro non è definito.**

Strategia di test ..... **Errore. Il segnalibro non è definito.**

Uso dell'Intelligenza Artificiale ..... **Errore. Il segnalibro non è definito.**

## Tema del progetto e introduzione

Il progetto ha lo scopo di simulare sistemi complessi modellando il comportamento collettivo di entità autonome, ossia i boids. Tale modello riproduce la dinamica e la cinematica di stormi di uccelli, banchi di pesci o sciami di insetti, tramite la regolazione del moto di ogni boid sulla base di regole locali, quali i fattori di separazione, allineamento e coesione.

Il codice che abbiamo implementato è stato suddiviso in file specializzati, ovvero: "main.cpp", "boids_logic.hpp", "boids_logic.cpp" e "boids.test.cpp". La logica del programma è stata posta nei due file boids_logic: nell'header (ovvero hpp) sono state inserite le varie dichiarazioni in seguito definite nel rispettivo file cpp. Quindi in questi file sono state definite le struct Boid, Boid_vel e Boid_complete, con le rispettive regole del moto implementate dentro la classe principale Movement. In essa sono quindi stati definiti tutti i parametri necessari alla caratterizzazione del moto dei boids che poi verranno richiesti in input. Le tre regole, chiamate rule1, rule2 e rule3 sono rispettivamente delle funzioni che sfruttando i parametri richiesti aggiornano la velocità di un boid basandosi su dei criteri di separazione, allineamento e coesione. Il metodo principale update(…) applica le regole tra i vari boids e ne aggiorna le posizioni. Su questi file sono stati eseguiti dei test per verificare il corretto funzionamento della logica del programma nel file "boids.test.cpp" utilizzando "doctest.h". Nel file main.cpp vengono richiesti in input i parametri del programma all'utente, si verifica che essi siano conformi alle regole definite utilizzando dei throw e dei catch. Sempre nel main la grafica è stata gestita tramite l'utilizzo della libreria SFML (*Simple and Fast Multimedia Library*), che consente la visualizzazione animata e l'interazione con la simulazione dei boids. In particolare, viene creata a runtime una finestra grafica 1600x900 pixel dove viene disegnata la simulazione, con limitazione del framerate a 90 FPS. La finestra viene aggiornata ad ogni frame tramite un ciclo while che in aggiunta aggiorna anche le posizioni e le velocità dei boids.

Essi vengono rappresentati da puntini bianchi/rossi inizializzati con posizioni e velocità casuali, invece attorno al cursore viene visualizzata una circonferenza colorata che rappresenta la zona d'influenza entro cui i boids risentono della forza esercitata dal puntatore. Se il tasto sinistro non è premuto, la forza è attrattiva e la circonferenza è verde; se, invece, il tasto sinistro è premuto costantemente, la forza diventa repulsiva e la circonferenza rossa. Questa interazione è realizzata mediante il metodo set_mouse_force(...) della classe Movement, che aggiorna i parametri interni relativi alla posizione del mouse, allo stato della pressione e alla modalità della forza. Il comportamento fisico viene poi gestito nel metodo update(...), in cui la forza del mouse viene applicata all'interno di un raggio prefissato tramite il metodo privato mouse_force(...).

### Librerie esterne da installare

Abbiamo utilizzato due librerie esterne, ovvero:

- SFML per l'interfaccia grafica;
- CMake per la compilazione.

## Istruzioni per eseguire il programma

Per eseguire il programma si sfrutta il sistema di build CMake.
Descriviamo i passaggi per ottenere due eseguibili, che avranno il nome di "boids_sim" e "boids_sim.t", rispettivamente per la rappresentazione grafica con in input i parametri per la gestione delle regole del volo e per la verifica dei test inseriti nel file "boids.test.cpp" sfruttando "doctest.h".
All'interno della cartella "build" lanciare il comando "cmake -G Ninja -DCMAKE_BUILD_TYPE=Debug .." per compilarlo in modalità debug, altrimenti "cmake -G Ninja -DCMAKE_BUILD_TYPE=Release .." per la modalità release. Una volta fatto ciò tramite il comando "ninja" si creano gli eseguibili citati precedentemente. A questo punto è sufficiente digitare "./boids_sim.t" per effettuare i test oppure "./boids_sim" per avviare il programma con la rappresentazione grafica.

## Parametri di input e formato output

All'avvio della simulazione il programma richiede all'utente i parametri da inserire per la regolazione del comportamento cinematico e dinamico del sistema. I parametri richiesti sono elencati in seguito e sono accompagnati da un ipotetico valore standard da potergli assegnare.
*Input da terminale*:
- Numero di boids "n_b" (es. 150): è un intero positivo che definisce il numero di boids presenti nel sistema;
- Distanza di interazione "d" (es. 50): è un numero positivo che definisce la distanza per la quale un boids inizia a risentire delle interazioni dovute alla presenza degli altri boids;
- Distanza di separazione "d_s" (es. 15): è un numero positivo che definisce la distanza massima di avvicinamento tra due boids prima che si attivi la regola di separazione;
- Coefficienti di separazione "s" (0.05): regola l'intensità della forza repulsiva;
- allineamento "a" (es. 0.05): regola l'intensità della forza di allineamento;
- coesione "c" (es. 0.01): regola l'intensità dell'attrazione dei boids verso il centro di massa;

*Il programma tornerà in output:*
- Finestra 1600x900 con animazione dei boids;
- Boid disegnati come cerchi, colore più rosso = maggiore velocità;
- Cerchio interattivo attorno al mouse se attiva la forza (verde = attrattiva, rosso = repulsiva).

## Interpretazione dei risultati

La simulazione mostra la formazione spontanea di stormi; i boids tendono ad avvicinarsi ad un centro di massa locale grazie al parametro "d" definito in precedenza, formando uno stormo all'interno del quale seguono traiettorie simili, coerentemente con le regole di separazione, allineamento e coesione. La variazione del colore dei boids è indice del modulo della velocità, maggiore è la velocità del boid maggiore è l'intensità del rosso. L'interazione col mouse ha evidenziato la corretta applicazione della forza attrattiva o repulsiva, con risposta immediata e stabile da parte dei boids.

## Strategia di test

Per garantire l'affidabilità e la correttezza del comportamento simulato, abbiamo integrato una lista di test automatici tramite Doctest. I test sono stati progettati per verificare le funzioni principali del progetto, valutandone l'output in diversi scenari, sia nominali che casi limite.
Abbiamo adottato dei test con l'obiettivo di verificare la correttezza di:
- Funzioni matematiche di supporto ("diff_pos", "velocity", "+=", "cm", "is_neighbor");
- Regole di movimento dei boids ("rule1", "rule2", "rule3");
- Aggiornamento dinamico ("update");
- Validazione degli input e limiti logici ("limit_velocity").

## Uso dell'Intelligenza Artificiale

Abbiamo utilizzato suggerimenti provenienti da strumenti di Intelligenza Artificiale generativa per ottimizzare alcune parti del codice, come la gestione grafica con SFML, l'interazione con il mouse e alcune ottimizzazioni sintattiche tramite il ternary conditional operator.